Unit 4

Q1) Demand Paging

Demand Paging is a memory management technique in which a page is loaded into memory only when it is required by the executing process, rather than preloading all pages into memory. This approach helps to save memory and improve system efficiency, as only the necessary parts of a program are loaded.

1. Components in Demand Paging

1. **Page Table**:
   Keeps track of whether a page is in memory or on disk.
   Each entry has a **valid/invalid bit**:
   Valid (1): Page is in memory.
   Invalid (0): Page is on disk.
2. **Secondary Storage (Disk)**:
   Stores pages that are not in memory.
   Acts as the backup for virtual memory.
3. **Main Memory (RAM)**:
   Holds the pages that are actively in use.

2. Steps in Demand Paging

Step 1: Process Requests a Page

A process makes a reference to a memory location.
This location corresponds to a page in virtual memory.

Step 2: Page Table Check

The **page table** is checked to determine whether the requested page is in memory.
If the page is present (**valid bit = 1**), it is accessed directly (**Page Hit**).
If the page is absent (**valid bit = 0**), a **Page Fault** occurs.

Step 3: Handling a Page Fault

When a page fault occurs:

1. **Suspend the Process**: The operating system halts the process temporarily.
2. **Locate the Page on Disk**: The OS identifies the location of the required page in secondary storage.
3. **Find a Free Frame in Memory**:
   If there is a free frame in memory, the page is loaded into it.
   if no free frame is available, a **page replacement algorithm** is used (e.g., FIFO, LRU) to avoid an existing page.
4. **Load the Page**: The OS transfers the page from the disk to memory.
5. **Update the Page Table**: The page table entry for the loaded page is updated, setting the **valid bit = 1**.

Step 4: Resume the Process

After loading the page, the OS resumes the process from where it was interrupted.

3. Advantages of Demand Paging

Efficient Memory Usage: Only required pages are loaded, saving memory space.
Faster Program Startup: Programs start quickly without preloading all pages.
Support for Large Programs: Enables execution of programs larger than physical memory.

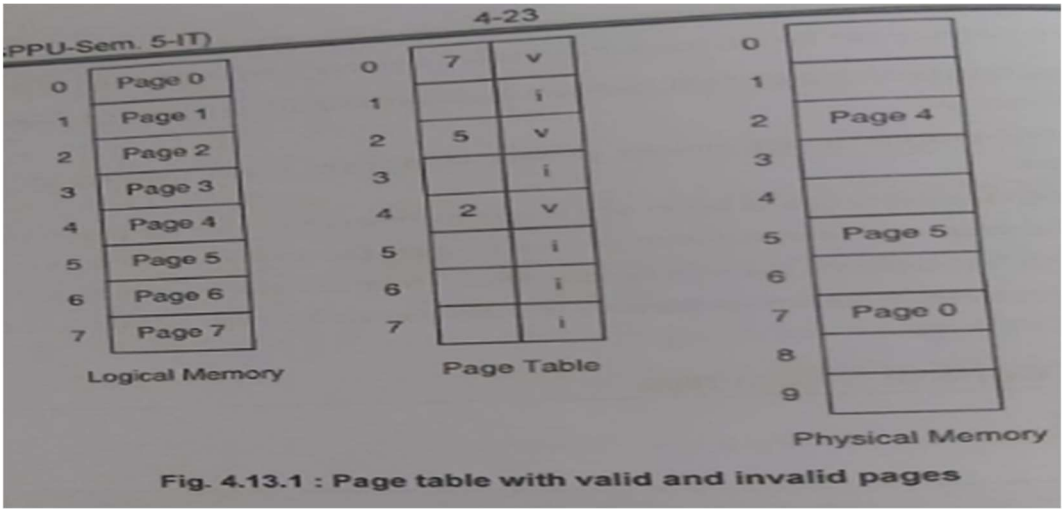


Fig. 4.13.1 : Page table with valid and invalid pages

Q2) **Buddy System**

The **Buddy System** is a memory allocation technique used in operating systems to manage memory in an efficient and structured manner. It divides memory into blocks of sizes that are powers of 2 and uses a binary tree-like approach for allocation and deallocation.

1. Key Concepts

1. **Memory Pool**: A contiguous block of memory is divided into smaller blocks called **buddies**.
2. **Division Rule**: Memory is always divided into two equal parts, with sizes being powers of 2 (e.g., $2^0, 2^1, 2^2, \ldots$).
3. **Buddy Pair**:

   Two blocks of the same size are called **buddies**. For example:

   If a block of size 242^424 is split into two blocks of 232^323, they are **buddies**.
4. **Coalescing**: When a block is deallocated, the system checks if its buddy is free. If both are free, they are combined into a larger block.

2. Steps in Buddy System

Allocation

1. A request for a block of size SSS is made.
2. The system finds the smallest block B where B ≥ SB.
3. If B > S , the block is split repeatedly into two equal halves until the desired size is obtained.

Deallocation

1. When a block is freed, the system checks its buddy.
2. If the buddy is also free, they are merged (coalesced) to form a larger block.
3. This process continues recursively until no further merger is possible.

3. Advantages of Buddy System

Efficient Allocation: Handles memory fragmentation effectively.

Easy Merging: Buddy blocks can be quickly merged.

Scalability: Suitable for dynamic memory allocation.

Example

1. Initial Memory: Total size = 128 KB.
2. Request 20 KB:

   Find the smallest block ≥ 20: 32 KB.

   Split 128 KB → 64 KB → 32 KB.

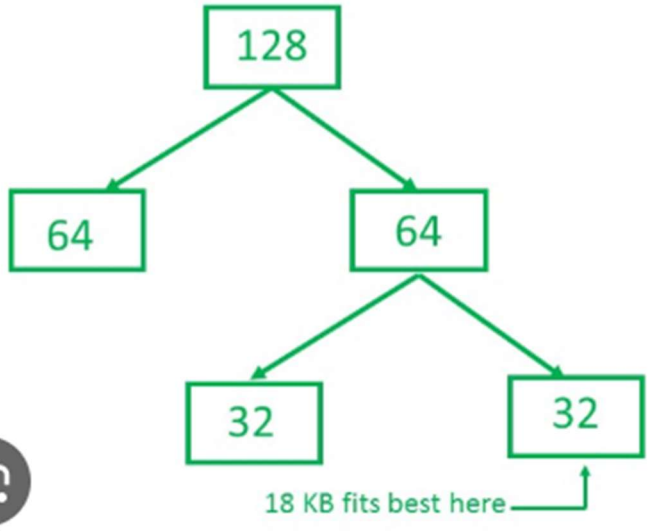
   Allocate 32 KB.
3. Request 50 KB:

   Find the smallest block ≥ 50: 64 KB.

   Allocate 64 KB.
4. Free 32 KB

   Buddy block (another 32 KB) is free.

   Coalesce to 64 KB.



Q3) What is segmentation? How address Translation is performed in segmentation system?

Segmentation is a memory management technique in which a program is divided into variable-sized logical units called segments. Each segment represents a logical group of related information, such as code, data, stack, etc.

1. **Key Features:**
   - Each segment has a name and a length.
   - Segments are mapped to different areas of physical memory.
   - The system maintains a Segment Table for each process to keep track of segments.
2. **Advantages:**
   - Reflects the logical view of a program.
   - Reduces external fragmentation.
   - Provides isolation between different segments.

Address Translation in Segmentation

Address translation in a segmentation system converts logical addresses (provided by a program) into physical addresses (used by memory).

1. **Logical Address:**
   Given as (Segment Number, Offset):
   - Segment Number (s): Identifies the segment in the segment table.
   - Offset (d): Specifies the location within the segment.
2. **Segment Table:**
   Each entry contains:
   - Base Address: Starting physical address of the segment in memory.
   - Limit: Length of the segment.
3. **Translation Process:**
   When a program accesses a memory location:
   1. **Segment Table Lookup**: The Segment Number (s) is used to find the base address and limit.
   2. **Check for Valid Offset**:
      - The Offset (d) is compared with the segment's limit.
      - If d < Limit, the offset is valid.
      - Otherwise, a segmentation fault occurs.
   3. **Compute Physical Address:** Physical Address = Base Address + Offset (d) **.**

Example

1. **Segment Table**:

Segment Number	Base Address	Limit
0	1000	400
1	2000	600
2	3000	300

2. **Logical Address**: (1, 120)
   Segment 1:
      Base Address = 2000.
      Limit = 600.
   Offset 120 < 600 → Valid.
   Physical Address = 2000+120=2120 .
3. **Logical Address**: (2, 350)
   Segment 2:
      Base Address = 3000.
      Limit = 300.
   Offset 350>300 → Invalid (Segmentation Fault).

Q4) **Belady's Anomaly**

Belady's anomaly is a counterintuitive phenomenon in **page replacement algorithms** where increasing the number of page frames in memory results in an **increase in the number of page faults**. This behavior is specific to certain algorithms, such as **First-In-First-Out (FIFO)**, and does not occur in optimal or Least Recently Used (LRU) algorithms.

Understanding Belady's Anomaly

1. **Page Fault**: Occurs when a page needed by a process is not in memory and must be loaded from disk.
2. **Expected Behavior**: Generally, increasing the number of page frames should reduce page faults because more pages can be stored.
3. **Belady's Anomaly**: In FIFO, increasing page frames can lead to more page faults due to the algorithm's simple "first-in, first-out" approach.

How Belady's Anomaly Works

1. Step-by-Step Process: Consider a reference string, which is a sequence of page requests. For each frame size (i.e., number of frames in memory), we simulate the page replacement process using the FIFO algorithm and track the number of page faults.

2. FIFO Replacement: When the memory is full and a new page is needed, FIFO will replace the page that has been in memory the longest, regardless of whether it's needed soon again.

Example : FIFO example and try with 3 frames and 4 frames

Q5) Page Fault

A page fault is an event that occurs when a program tries to access a page (a fixed-length block of memory) that is not currently in the main memory (RAM). When the program attempts to access a page that is not in memory, the operating system (OS) must handle the fault by loading the page from secondary storage (typically the hard drive or SSD) into memory.

Types of Page Faults:

1. **Minor Page Fault**: Occurs when the page is not in memory but is still available in the **swap space** or **disk cache**. This can be resolved quickly without having to go to the disk.
2. **Major Page Fault**: Occurs when the page is not in memory, and it needs to be fetched from the **hard disk** or **SSD**, which is a slower process.

Q6) What are the distinctions among logical, relative and physical addresses?

Aspect	Logical Address	Relative Address	Physical Address
Definition	Address generated by the CPU during execution.	Address relative to the segment's base address.	Actual address in physical memory (RAM).
Generated by	The program or CPU.	Based on a segment's base address (used in segmentation).	The memory management unit (MMU) after translation.
Used by	Programs in virtual memory systems.	Mainly used in segmentation (for memory segments).	Hardware (CPU) to access actual data in RAM.
Translation	Translated by the MMU into a physical address.	Converted into a logical address by adding the base address.	Directly corresponds to a location in physical memory.
Example	`0x00400000` (in virtual address space).	`0x1000` relative to a segment with a base of `0x2000`.	`0x1A000000` (actual location in RAM).

Q7) Concept of Virtual Memory

Virtual Memory is a memory management technique used by modern operating systems to provide an "idealized abstraction" of the storage resources that are actually available on a given machine. It creates the illusion for users and programs that they have access to a large, continuous block of memory, even if the physical memory (RAM) is smaller and fragmented.

Virtual memory allows programs to run without worrying about how much physical memory is available. It does this by using both **RAM** and a portion of **secondary storage** (such as a hard drive or SSD), which is treated as if it were part of the computer's RAM.

Key Features of Virtual Memory:

1. **Separation of Logical and Physical Memory**:

Logical memory refers to the address space used by programs, while **physical memory** refers to the actual memory hardware (RAM).

Virtual memory enables a separation, where programs are given the illusion of accessing a large contiguous block of memory, even though only a small portion of it might be stored in physical memory at any time.

2. **Paging and Segmentation**:

Virtual memory uses techniques like **paging** and **segmentation** to manage memory more efficiently:

Paging: Memory is divided into fixed-size blocks called **pages**. The operating system keeps track of all these pages and can move them between physical memory and disk storage.

Segmentation: Memory is divided into different segments (e.g., code, data, stack), and each segment can be allocated independently in physical memory.

Address Translation:

Programs use **virtual addresses** to access memory. The **Memory Management Unit (MMU)** of the CPU translates these virtual addresses into **physical addresses** in RAM or swap space.

Swap Space:

When there is insufficient physical memory, the operating system uses **swap space** (usually located on the disk) to store inactive pages of memory. This is known as **paging out** or **swapping**. When a program needs a page that isn't in RAM, the OS swaps it back into memory.

Demand Paging:
Virtual memory supports **demand paging**, which means that a page of memory is only loaded into RAM when it is actually needed by the program, not when it is first referenced. This reduces the overall memory requirements of running applications.

Example of Virtual Memory in Action:
Suppose a system has 4 GB of RAM, but a program requests 10 GB of memory. With virtual memory:

The operating system can divide the requested 10 GB into smaller chunks or **pages**.

These pages can be stored in physical memory as well as in swap space (on the disk).

When the program needs access to a page that isn't in RAM, a **page fault** occurs, and the OS swaps the required page into memory from the disk.

The program continues execution as if it is using the full 10 GB of memory, even though it might only be utilizing a small portion of the physical memory at any given time.

---

n